МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
<БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ>

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность АС-67

Выполнил
А. И. Петрова,
студент группы АС-67

Проверил
А. А. Крощенко,
ст. преп. кафедры ИИТ,
«___> _____2026 г.

Брест 2026

Цель: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации

Вариант 7 Прогнозирование оттока клиентов

- Telco Customer Churn
- **Задача:** предсказать, уйдет ли клиент (бинарная классификация).
- **Архитектура:**
  - входной слой;
  - один скрытый слой с 24 нейронами (ReLU);
  - выходной слой с 1 нейроном (Sigmoid).
- **Эксперимент:** добавьте в модель слой Dropout с коэффициентом 0.2 после скрытого слоя и сравните результаты. Помогло ли это?

Выполнение:

Код программы

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix
)

torch.manual_seed(42)
np.random.seed(42)

print("   Запуск задачи прогнозирование (Telco Customer Churn)")

df = pd.read_csv("Telco-Customer-Churn.csv")

df["TotalCharges"] = pd.to_numeric(
    df["TotalCharges"].astype(str).str.strip(),
    errors="coerce"
)

X = df.drop(["customerID", "Churn"], axis=1)
y = (df["Churn"] == "Yes").astype(int)

print("Количество записей:", len(df))
print("Количество признаков:", X.shape[1])
print("Клиенты без оттока:", (y == 0).sum())
print("Клиенты с оттоком:", (y == 1).sum())
print("Пропуски TotalCharges:", X["TotalCharges"].isna().sum())

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

numeric_features = X.select_dtypes(
```

```
    include=["number"]
).columns.tolist()

categorical_features = X.select_dtypes(
    exclude=["number"]
).columns.tolist()

for column in numeric_features:
    median = X_train[column].median()
    X_train.loc[:, column] = X_train[column].fillna(median)
    X_test.loc[:, column] = X_test[column].fillna(median)

for column in categorical_features:
    mode = X_train[column].mode()[0]
    X_train.loc[:, column] = X_train[column].fillna(mode)
    X_test.loc[:, column] = X_test[column].fillna(mode)

X_train = pd.get_dummies(
    X_train,
    columns=categorical_features,
    drop_first=True,
    dtype=int
)

X_test = pd.get_dummies(
    X_test,
    columns=categorical_features,
    drop_first=True,
    dtype=int
)

X_test = X_test.reindex(
    columns=X_train.columns,
    fill_value=0
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)

y_train = torch.tensor(
    y_train.to_numpy().reshape(-1, 1),
    dtype=torch.float32
)

y_test = torch.tensor(
    y_test.to_numpy().reshape(-1, 1),
    dtype=torch.float32
)

class MLP(nn.Module):
    def __init__(self, input_size, dropout=False):
        super().__init__()

        layers = [
            nn.Linear(input_size, 24),
            nn.ReLU()
        ]

        if dropout:
            layers.append(nn.Dropout(0.2))

        layers.append(nn.Linear(24, 1))
```

```
        self.layers = nn.Sequential(*layers)

    def forward(self, x):
        return self.layers(x)

def train_model(dropout, name):
    torch.manual_seed(42)

    model = MLP(X_train.shape[1], dropout)

    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.Adam(
        model.parameters(),
        lr=0.001
    )

    for epoch in range(1, 201):
        model.train()

        output = model(X_train)
        loss = criterion(output, y_train)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if epoch % 50 == 0:
            print(f"{name} | Эпоха {epoch}, Loss: {loss.item():.4f}")

    model.eval()

    with torch.no_grad():
        output = model(X_test)
        probabilities = torch.sigmoid(output)
        predictions = (probabilities >= 0.5).int().numpy().flatten()

    actual = y_test.numpy().flatten()

    accuracy = accuracy_score(actual, predictions)
    precision = precision_score(actual, predictions, zero_division=0)
    recall = recall_score(actual, predictions, zero_division=0)
    f1 = f1_score(actual, predictions, zero_division=0)

    cm = confusion_matrix(actual, predictions)

    print(f"\n{name}")
    print(f"Accuracy:  {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall:    {recall:.4f}")
    print(f"F1-score:  {f1:.4f}")

    print("\nМатрица ошибок:")
    print(cm)

    return {
        "Модель": name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-score": f1
    }

print("\nОБУЧЕНИЕ МОДЕЛИ БЕЗ DROPOUT")
result_1 = train_model(
    False,
    "MLP без Dropout"
)
```

```
print("\nОБУЧЕНИЕ МОДЕЛИ С DROPOUT 0.2")
result_2 = train_model(
    True,
    "MLP с Dropout 0.2"
)

print("\nСРАВНЕНИЕ РЕЗУЛЬТАТОВ")

results = pd.DataFrame([result_1, result_2])

print(results.to_string(
    index=False,
    float_format=lambda x: f"{x:.4f}"
))

print("\nИЗМЕНЕНИЕ МЕТРИК")

print(f"Изменение Precision: {result_2['Precision'] - result_1['Precision']:+.4f}")
print(f"Изменение Recall: {result_2['Recall'] - result_1['Recall']:+.4f}")
print(f"Изменение F1-score: {result_2['F1-score'] - result_1['F1-score']:+.4f}")

if result_2["F1-score"] > result_1["F1-score"]:
    print("Добавление Dropout улучшило F1-score.")
elif result_2["F1-score"] < result_1["F1-score"]:
    print("Добавление Dropout снизило F1-score.")
else:
    print("Dropout не изменил F1-score.")
```

Результат:

Запуск задачи прогнозирования (Telco Customer Churn)
Количество записей: 7043
Количество признаков: 19
Клиенты без оттока: 5174
Клиенты с оттоком: 1869
Пропуски TotalCharges: 11

ОБУЧЕНИЕ МОДЕЛИ БЕЗ DROPOUT
MLP без Dropout | Эпоха 50, Loss: 0.4785
MLP без Dropout | Эпоха 100, Loss: 0.4351
MLP без Dropout | Эпоха 150, Loss: 0.4221
MLP без Dropout | Эпоха 200, Loss: 0.4153

MLP без Dropout
Accuracy: 0.7885
Precision: 0.6188
Recall: 0.5294
F1-score: 0.5706

Матрица ошибок:
[[913 122]
 [176 198]]

ОБУЧЕНИЕ МОДЕЛИ С DROPOUT 0.2
MLP с Dropout 0.2 | Эпоха 50, Loss: 0.4837
MLP с Dropout 0.2 | Эпоха 100, Loss: 0.4404
MLP с Dropout 0.2 | Эпоха 150, Loss: 0.4301
MLP с Dropout 0.2 | Эпоха 200, Loss: 0.4240

MLP с Dropout 0.2
Accuracy: 0.7885
Precision: 0.6180
Recall: 0.5321
F1-score: 0.5718

Матрица ошибок:
[[912 123]
 [175 199]]

СРАВНЕНИЕ РЕЗУЛЬТАТОВ

Модель	Accuracy	Precision	Recall	F1-score
MLP без Dropout	0.7885	0.6188	0.5294	0.5706
MLP с Dropout 0.2	0.7885	0.6180	0.5321	0.5718

ИЗМЕНЕНИЕ МЕТРИК
Изменение Precision: -0.0007
Изменение Recall: +0.0027
Изменение F1-score: +0.0012

Добавление Dropout улучшило F1-score.

Вывод: В ходе лабораторной работы была разработана и обучена нейронная сеть многослойный перцептрон (MLP) для прогнозирования оттока клиентов телекоммуникационной компании. Были реализованы и сравнены две модели: первая с одним скрытым слоем из 24 нейронов и вторая с дополнительным слоем Dropout с коэффициентом 0,2. В результате эксперимента Accuracy обеих моделей составила 0,7885. После добавления Dropout значение Recall увеличилось с 0,5294 до 0,5321, а F1-score — с 0,5706 до 0,5718. При этом Precision незначительно снизилась с 0,6188 до 0,6180. Таким образом, добавление Dropout привело к небольшому улучшению F1-score и Recall, однако существенного изменения качества классификации не произошло.